FINAL PROJECT REPORT

AuraFlex Gym Management System

Group Number : 11

Name	Index Number
S.H.Chehan Mendiya	TG/IT/2025/2500
K.R.Karanayaka	TG/IT/2025/2498
R.Indumin Sasindu	TG/IT/2025/2493
T.V.K.Sendiya	TG/2024/2120
Pasindu Madusanka	TG/IT/2025/2485

Declaration

We declare that this report and the AuraFlex Gym Management System are the original work of our group. The source code, the design, the test data, and the documents were prepared by our group for this project. We used external documentation only to understand standard C features such as library functions and file handling, and these sources are listed in the References section.

Acknowledgement

We would like to thank our lecturer Ms. E.H.M.P.M. Wijerathna and demonstrators for their guidance and feedback during the requirements, design, coding, and testing stages of this project. We also used the documentation of the standard C library and the compiler to solve technical problems about console input, file processing, date and time handling, and string validation.

Abstract

AuraFlex is a menu-driven gym management system written in the C programming language. A gym has to manage members, memberships, payments, attendance, trainers, and reports. When this is done manually, it often leads to duplicate records, lost payment details, and missed membership expiry dates. AuraFlex solves this problem by keeping all this work inside one console application.

The system has a role-based interface for administrators, receptionists, and trainers. Users log in with a username and password that are checked against a user file, and the dashboard depends on their role. The program has five main components: access, member and membership management; payment management; attendance management; trainer management; and reports. It uses modular C source and header files, structures to hold records, helper functions for validation, and text files for permanent storage. Separate files store the users, members, memberships, payments, attendance records, trainers, and trainer-member assignments. Temporary files are used when a record must be updated or removed safely.

The main features are masked password entry, a limit on login attempts, duplicate ID prevention, member status control, membership assignment with payment processing, automatic payment and attendance IDs, date and time recording, check-in and check-out control, trainer-member assignment, payment totals, expired membership detection, and daily attendance summaries. Input and file errors are shown to the user.

We tested the menus, record validation, file saving, role-based navigation, and the link between membership and payment processing. The system shows that a small gym can manage its daily work without a database server. Its main limitations are plain text storage, limited search options, terminal-only use, and a password input method that depends on the platform. These can be improved in the future with a database, stronger password protection, better reports, and a graphical interface.

Table of Contents

Declaration **2**
Acknowledgement **2**
Abstract **2**
Table of Contents **3**
1. Introduction **4**
1.1 Background and Problem Description 4
1.2 Project Aim and Objectives 4
1.3 Scope of the System 4
1.4 Development Environment 5
2. Requirement Summary **5**
2.1 Users and Main Operations 5
2.2 Functional Components 5
2.3 Major Validation 6
3. System Design and Architecture **6**
3.1 Overall System Architecture 6
3.2 Menu Structure / Navigation 7
4. Methodology **8**
4.1 Development Approach 8
4.2 Programming Techniques Used 8
4.3 Integration Process 8
5. Implementation **9**
5.1 Component 1 - Access, Member and Membership Management 9
5.2 Component 2 - Payment Management 9
5.3 Component 3 - Attendance Management 9
5.4 Component 4 - Trainer Management 10
5.5 Component 5 - Reports 10
6. Testing and Results **10**
7. Problems Faced During Development **11**
8. Conclusion and Future Improvements **12**
9. References **12**
10. Appendices **13**
Appendix A - Source Structure 13
Appendix B - Main Record Formats 13
Appendix C - Example Workflows 14

1. Introduction

1.1 Background and Problem Description

A gym has to handle many related activities at the same time. These include registering members, tracking membership periods, collecting fees, recording attendance, giving trainers to members, and preparing summary reports. When these jobs are done using notebooks or separate files, staff can create duplicate records, lose payment information, miss membership expiry dates, or record attendance wrongly. It is also hard to control which staff member is allowed to do which task.

AuraFlex solves this problem with one central terminal application. The system keeps its records in structured text files and gives menus for the main gym activities. A computerized system is useful because it checks IDs, checks the member status, generates record IDs, reuses common functions, and shows summaries from the stored records, so the work becomes more consistent.

1.2 Project Aim and Objectives

The aim of this project is to develop a modular gym management system in C that supports reliable day-to-day gym administration.

The objectives of the project are to:

- authenticate users and give role-based access;
- register, search, update, and manage member records;
- assign and display memberships and membership fees;
- record payments and search the payment history;
- track the check-ins and check-outs of members;
- manage trainers and trainer-member assignments;
- generate membership, payment, attendance, and system summary reports; and
- validate user input and protect the stored records during updates.

1.3 Scope of the System

The system covers:

- administrator, receptionist, and trainer roles;
- user login with an active-status check;
- member registration and status management;
- membership assignment and payment recording;
- attendance check-in, check-out, history, and daily summaries;
- trainer registration, search, update, status changes, and member assignment;
- member, membership, expired membership, payment, and summary reports; and
- permanent storage using text files.

The system does not include: online payments, a database server, email or SMS notifications, biometric attendance, a graphical interface, access from several computers at the same time, advanced audit reports, and detailed role permissions for every operation.

Limitation: The fields in the text files are separated by spaces, so names and plan descriptions must be one word. Underscores can be used when more than one word is needed.

1.4 Development Environment

Category	Description
Programming Language	C
Operating Environment	Terminal / command-line environment. The authentication module uses Windows console input through `conio.h`.
Interface	Menu-driven text interface with formatted console tables
Data Storage	Plain text files in the `data/` folder
Compiler	A C compiler such as GCC, Clang, or the compiler of an IDE

2. Requirement Summary

This section gives a short summary of the final requirements. The system is a small gym system with authenticated access, record management, validation, permanent storage, and summary output. It is divided into modules, so each major requirement could be developed and tested separately.

2.1 Users and Main Operations

Administrator

- Manage members, memberships, trainers, payments, and attendance.
- View system reports and summaries.
- Use the complete administrator dashboard.

Receptionist

- Register and view members.
- Manage memberships.
- Record payments.
- Record attendance.

Trainer

- View assigned members.
- Assign members to trainers.
- Remove member assignments.

2.2 Functional Components

The system has the following functional components:

1. Authentication and role-based dashboard access.

2. Member and membership management.
3. Payment management.
4. Attendance management.
5. Trainer management.
6. Reports and summaries.

2.3 Major Validation

The system validates the following:

- user credentials and active user status;
- the maximum number of login attempts;
- duplicate member, membership, trainer, payment, and attendance IDs;
- existing and active member IDs;
- positive payment and membership amounts;
- payment dates in the `YYYY-MM-DD` format;
- accepted payment methods, which are `CASH`, `CARD`, and `ONLINE`;
- duplicate check-ins on the same day;
- an open check-in record before check-out;
- an active trainer before a member is assigned;
- numeric menu input and wrong menu selections; and
- file opening, writing, temporary file, and file replacement errors.

3. System Design and Architecture

3.1 Overall System Architecture

The system uses a modular, file-based design. The file `main.c` is the entry point of the program, and it controls the login and the dashboard navigation. The header files declare the public functions of each module, and the source files contain the code. The data files store the records permanently. Figure 1 shows the components, the shared files, and the main data flows.

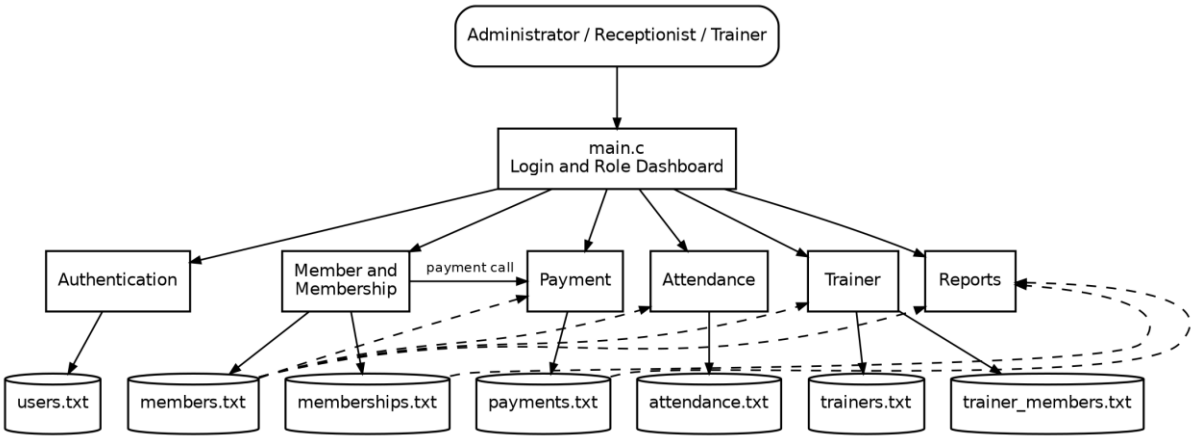


Figure 1: Overall system architecture (dashed arrows show data read from shared files)

Examples of data flow

- Login reads `users.txt`, returns the role of the user, and opens the correct dashboard.

- Membership assignment checks `members.txt`, saves the membership, and calls the payment module.
- Attendance checks `members.txt` before it writes to `attendance.txt`.
- Reports read the members, memberships, and payments to calculate totals and status summaries.
- Trainer assignment checks `trainers.txt` and `members.txt` before it writes to `trainer_members.txt`.

3.2 Menu Structure / Navigation

The main menu of each role and the important submenus are shown below.

```
Login
+-- Administrator Dashboard
|   +-- Member Management
|   +-- Membership Plans / Membership Management
|   +-- Trainer Management
|   +-- Payment Management
|   +-- Attendance Management
|   +-- Reports
|   +-- Logout
+-- Receptionist Dashboard
|   +-- Register Member
|   +-- Manage Membership
|   +-- Record Payment
|   +-- Record Attendance
|   +-- View Members
|   +-- Logout
+-- Trainer Dashboard
    +-- View Assigned Members
    +-- Assign Member
    +-- Remove Member
    +-- Logout
```

Important submenus:

```
Payment Management
+-- Record Payment
+-- Search Payment History
+-- View All Payments
+-- Calculate Total Payments
+-- Return

Attendance Management
+-- Record Check-In
+-- Record Check-Out
+-- Search Attendance by Member ID
+-- View Attendance History
+-- Daily Attendance Summary
+-- Return

Reports
+-- Member Report
+-- Membership Report
+-- Expired Membership Report
+-- Payment Report
+-- Summary Report
+-- Return
```

4. Methodology

4.1 Development Approach

Our group used a modular development approach. First, the requirements were divided according to the main gym operations. Each member of the group developed one or more related modules, including the header declarations, the source code, and the data files for that module. The functions were built around small record structures and file helper functions, so each module could be tested on its own.

After the individual work, the modules were connected through `main.c` and the shared header files. During integration we focused on role-based navigation, member checking, membership payment processing, attendance validation, trainer assignments, and the reports that use data from several modules. We tested with valid and invalid menu inputs, existing and non-existing IDs, status changes, file operations, and workflows that use more than one component.

4.2 Programming Techniques Used

The following programming concepts were used to develop the system:

- **Procedural programming:** each operation is written as a separate C function.
- **Modular programming:** public functions are declared in `.h` files and the code is written in separate `.c` files.
- **Structures:** records such as `User`, `Member`, `Membership`, `Payment`, `Attendance`, and trainer records keep related fields together.
- **File processing:** `fopen`, `fscanf`, `fprintf`, `fgets`, `rename`, and `remove` are used to store data in text files.
- **Validation helpers:** reusable functions check IDs, status, dates, amounts, and menu input.
- **Sentinel value:** an open attendance record uses `-` as its check-out value.
- **Temporary file replacement:** updates and deletions are done by rewriting the records into a temporary file.
- **String and date processing:** `strcmp`, `strlen`, `strcpy`, `sscanf`, `strftime`, and `time` are used for validation and reports.
- **Role-based control flow:** the role of the logged-in user decides which dashboard and operations are shown.

The code below shows how the membership module uses the payment module:

```
if (!processMembershipPayment(m.memberId, m.fee)) {
    printf("Membership assignment cancelled because payment failed.\n");
    return;
}
```

4.3 Integration Process

The modules were integrated using the public function declarations in the header files. The file `main.c` calls the top-level menus, and the modules call each other's functions where a requirement needs more than one component. The most important integration points are:

- `login()` returns a filled `User` structure to `main.c`.
- `membership.c` calls `processMembershipPayment()` in `payment.c`.

- `payment.c` checks active members using `members.txt`.
- `attendance.c` checks active members before check-in and check-out.
- `trainer.c` checks members before it creates trainer assignments.
- `reports.c` reads the member, membership, and payment files to produce summaries.

5. Implementation

5.1 Component 1 - Access, Member and Membership Management

Authentication

The authentication module reads the user records from `data/users.txt`, compares the username and password that were entered, and accepts only active users. It allows three attempts and hides the password while it is typed. The record of the logged-in user is passed to the main dashboard, so the correct menu for the role can be selected.

Member management

Member management stores the member ID, name, phone number, email, and status. It supports registering, viewing, searching, updating, freezing, and unfreezing members. Duplicate IDs are rejected, and a temporary file is used when a record is updated.

Membership management

Membership management stores the membership ID, member ID, plan name, start date, end date, fee, and status. It checks that the member exists and that the membership ID is not already used. New memberships are marked as active. When a membership is assigned, the payment component is also called so that the fee is recorded.

Main files:

- `src/auth/auth.c`, `include/auth.h`
- `src/member/member.c`, `include/member.h`
- `src/membership/membership.c`, `include/membership.h`
- `data/users.txt`, `data/members.txt`, `data/memberships.txt`

5.2 Component 2 - Payment Management

The payment module stores the payment ID, member ID, amount, date, and payment method. It supports normal payment recording, membership payments, searching the payment history of a member, viewing all payments, and calculating the total amount.

Payment IDs are generated by reading the existing records and choosing the next free `P###` value. The module checks that the member exists and is active, that the amount is positive, and that the date is a valid calendar date, including leap years. Only the methods `CASH`, `CARD`, and `ONLINE` are accepted.

Main files:

- `src/payment/payment.c`, `include/payment.h`
- `data/payments.txt`

5.3 Component 3 - Attendance Management

The attendance module stores the attendance ID, member ID, date, check-in time, and check-out time. It gets the current date and time automatically. A member can check in only when the member exists, is active, and has no open attendance record for that day.

For check-out, the program writes all records to `attendance_temp.txt` and replaces the original file only after the writing is successful. The module also gives attendance search, the full history, and a daily summary. The daily summary counts the check-ins, the completed check-outs, and the members who are still inside.

Main files:

- `src/attendance/attendance.c`, `include/attendance.h`
- `data/attendance.txt`

5.4 Component 4 - Trainer Management

Trainer management stores the trainer ID, name, phone number, specialization, and status. It supports registering, viewing, searching, updating, activating, and deactivating trainers, and assigning members to trainers.

Assignments are stored as trainer ID and member ID pairs in `trainer_members.txt`. The module checks that the trainer exists and is active, that the member exists, and that the same assignment does not already exist. When an assignment is removed, the file is rewritten using a temporary file.

Main files:

- `src/trainers/trainer.c`, `include/trainer.h`
- `data/trainers.txt`, `data/trainer_members.txt`

5.5 Component 5 - Reports

The reports module provides the member, membership, expired membership, payment, and summary reports. The existing functions for viewing members and memberships are reused where possible. The expired membership report compares the end dates of active memberships with the current date. The payment report counts the records and calculates the total amount. The summary report combines the member, membership, and payment information.

Main files:

- `src/reports/reports.c`, `include/reports.h`
- `data/members.txt`, `data/memberships.txt`, `data/payments.txt`

6. Testing and Results

We tested the system at two levels. First each module was tested separately, and then the whole system was tested through the main menu. The test areas are shown in the table below.

Test area	Examples	Expected result
Login	Correct credentials, wrong credentials, inactive user, repeated failures	Valid active users enter the correct dashboard and invalid users are rejected
Members	New ID, duplicate ID, search, update, freeze / unfreeze	Records are created or updated correctly and duplicate IDs are blocked

Test area	Examples	Expected result
Memberships	Existing member, missing member, duplicate membership ID, positive fee	Only valid membership records are saved
Payments	Active member, frozen member, invalid amount, invalid date, invalid method	Invalid payments are rejected and valid payments are saved
Attendance	First check-in, duplicate check-in on the same day, check-out, no open check-in	Attendance status is kept correctly
Trainers	Registration, duplicate ID, status change, assignment, duplicate assignment	Trainer records and assignments stay consistent
Reports	Payment totals, expired dates, membership status counts	The summaries match the records read from the files
Integration	Membership assignment followed by payment, dashboard selection by role	Calls between modules and role-based navigation work together

The system shows clear messages for wrong input and file errors. Valid records are saved in the correct files, and the temporary file replacement protects the original data when a record is updated. The project folder also has compiled test files in the `build/` folder. More automated tests can be added when the system is extended.

7. Problems Faced During Development

Updating records in text files

Text files cannot be edited directly at one record. To solve this, we wrote a temporary copy with the changed record and renamed it to the original file name after the writing was successful.

Keeping the modules connected

A membership needs a valid member and a payment. Attendance and trainer assignment also depend on the member status. We used shared validation functions and public header declarations to keep these links consistent.

Duplicate IDs

IDs that are typed by the user can be duplicated. The member, membership, and trainer IDs are checked before they are saved, and the payment and attendance IDs are generated from the existing records.

Date and time handling

Membership expiry, payment dates, and attendance records all depend on dates and times. The system uses `time.h`, formatted dates, calendar validation, and string comparison for dates in the `YYYY-MM-DD` form.

Input and buffer handling

Mixing number input and text input can leave newline characters in the input buffer. The modules clear the buffer and limit the input size, but using `scanf` with text files is still a limitation.

Platform dependency

The authentication module uses `conio.h` and `_getch()` to hide the password, and this does not work on all operating systems. In a future version this should be replaced with an input method that works on every platform.

Data quality and security limitations

Plain text files are easy to open and read, and they do not give database transactions, control over several users at the same time, or encrypted passwords. The current login design is fine for a small academic project, but it should not be used in a real system without password hashing and stronger access control.

8. Conclusion and Future Improvements

AuraFlex shows a modular gym management system in C that handles the common administrative tasks of a gym. It logs in users, gives separate dashboards for each role, manages members and memberships, records payments and attendance, assigns trainers, and produces reports. The use of structures, helper functions, permanent files, validation, and temporary file replacement gives a good base for a small command-line application.

The following improvements can be made in the future:

- replace the text files with a relational database;
- hash the passwords and add safer credential management;
- replace the platform-specific password input;
- add stronger date validation to all membership operations;
- support names and descriptions that have spaces;
- add search by date, status, trainer, and payment method;
- add reports that can be exported as PDF or CSV;
- add audit logs and permission checks for sensitive operations;
- add automated unit and integration tests;
- add a graphical or web-based interface; and
- support several users at the same time, with backup and restore.

9. References

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[2] ISO/IEC, *Information technology — Programming languages — C*, ISO/IEC 9899:2018, 2018.

[3] Microsoft, “Console input and _getch function documentation,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/getch-getwch. [Accessed: Oct. 31, 2026].

[4] cppreference.com, “C file input/output,” “C time library,” and “C string library.” [Online]. Available: https://en.cppreference.com/w/c. [Accessed: Oct. 31, 2026].

[5] AuraFlex project source files and approved software requirements specification, group project materials, 2026.

10. Appendices

Appendix A - Source Structure

```
AuraFlex/
+-- data/
|   +-- attendance.txt
|   +-- members.txt
|   +-- memberships.txt
|   +-- payments.txt
|   +-- trainer_members.txt
|   +-- trainers.txt
|   +-- users.txt
+-- include/
|   +-- attendance.h
|   +-- auth.h
|   +-- member.h
|   +-- membership.h
|   +-- models.h
|   +-- payment.h
|   +-- reports.h
|   +-- system.h
|   +-- trainer.h
|   +-- ui.h
+-- src/
|   +-- attendance/attendance.c
|   +-- auth/auth.c
|   +-- member/member.c
|   +-- membership/membership.c
|   +-- payment/payment.c
|   +-- reports/reports.c
|   +-- system/system.c
|   +-- system/ui.c
|   +-- trainers/trainer.c
|   +-- main.c
+-- docs/
```

Appendix B - Main Record Formats

```
users.txt:
userId username password role status

members.txt:
memberId name phone email status

memberships.txt:
membershipId memberId planName startDate endDate fee status

payments.txt:
paymentId memberId amount paymentDate method

attendance.txt:
attendanceId memberId date checkIn checkOut

trainers.txt:
trainerId name phone specialization status

trainer_members.txt:
trainerId memberId
```

Appendix C - Example Workflows

Membership purchase

```
Login
-> Administrator or Receptionist Dashboard
-> Manage Membership
-> Assign Membership
-> Verify member
-> Enter membership details
-> Process membership payment
-> Save membership record
```

Attendance visit

```
Login
-> Attendance Management
-> Record Check-In
-> Verify active member
-> Generate attendance ID
-> Save current date and time
-> Record Check-Out
```

Trainer assignment

```
Login
-> Trainer Management
-> Assign Member to Trainer
-> Verify active trainer
-> Verify member
-> Prevent duplicate assignment
-> Save trainer/member relationship
```

GitHub Repository:

- https://github.com/ChehanMendiyaa/AuraFlex.git
- The repository contains the project source files, header files, sample data files, documentation, and other resources required for development and compilation.
- The repository also maintains the development history and contributions of the group members.